



Exploring Efficient FPGA Acceleration of High-Order 3D Iterative Stencil Loops on Large Data Grids

Alhussain Ibrahim¹ · Muhammad E. S. Elrabaa² · Saleh Alsaleh² · Aiman H. El-Maleh² · Thierry Tonellot³

Received: 20 July 2025 / Accepted: 18 November 2025
© King Fahd University of Petroleum & Minerals 2025

Abstract

An efficient methodology for FPGA acceleration of high-order 3D iterative stencil loops over large 3D-grids was developed. Spatial (tiling) and temporal (combined iterations) blocking are used to circumvent the FPGAs' limitations and maximize throughput. Implemented as a fully asynchronous SW-HW pipeline, it can compute high-order stencils on 3D grid without any limitations on the grid size or the number of iterations. An 8th-order, 25-point 3D stencil was used to demonstrate the methodology and possible optimizations of performance, resource utilization, and power efficiency. Results show that throughput is only limited by the FPGA off-chip memory bandwidth. Comparisons with published results for the same stencil showed that the developed methodology can achieve a throughput equivalent to ~ 43-Haswell cores (22 nm technology) or ~ 13 Milan-x cores (7 nm technology) running at 2.3 and 2.45 GHz, respectively. Compared to an A100 NVIDIA GPU implementation of the same stencil, it achieved ~ 41% better power efficiency (Watts per GB/s). The methodology was extended to support multiple FPGAs. With two FPGA boards, the total latency was reduced by ~ 27%.

Keywords Hardware implementation of stencil computations · High-performance computing · FPGA acceleration · High-order stencils

1 Introduction

Iterative stencil computations (i.e. iterative stencil loops) are commonly used in many high-performance computing

(HPC) applications. They involve iterative grid-based operations where each point on a grid is computed based on the values of its neighbors from the previous iteration. The positions and the number of neighboring elements needed for each computation varies depending on stencil's application. Figure 1 shows the data (memory) access pattern for a 3D stencil. Higher order stencils use more neighboring points (e.g. an 8th-order, 25-point stencil uses 4 points in each direction).

Data parallelism inherent in stencil computations makes them ideal for parallel computing. As such, there have been numerous implementations of different stencil computations (2D and 3D) in the literature on different platforms (multi-core CPUs, GPUs, and FPGAs). These implementations attempted to overcome some of the inherent challenges involved in stencil computations such as large memory footprint, low data-reuse (low arithmetic intensity), and highly irregular data access patterns. The latter becomes exponentially worse at higher dimensions.

Field Programmable Gate Arrays (FPGAs) have been increasingly utilized as accelerators in many applications due to their hardware configurability, fine-grain parallelism, and

✉ Muhammad E. S. Elrabaa
elrabaa@kfupm.edu.sa

Alhussain Ibrahim
aibrahim@palmlab.com

Saleh Alsaleh
salehs@kfupm.edu.sa

Aiman H. El-Maleh
aimane@kfupm.edu.sa

Thierry Tonellot
thierrylaurent.tonellot@aramco.com

¹ Palmlab, Riyadh, Saudi Arabia

² Computer Engineering Department and IRC-ISS, King Fahd University of Petroleum and Minerals (KFUPM), Dhahran, Saudi Arabia

³ Advanced Research Center, Saudi Aramco, Dhahran, Saudi Arabia



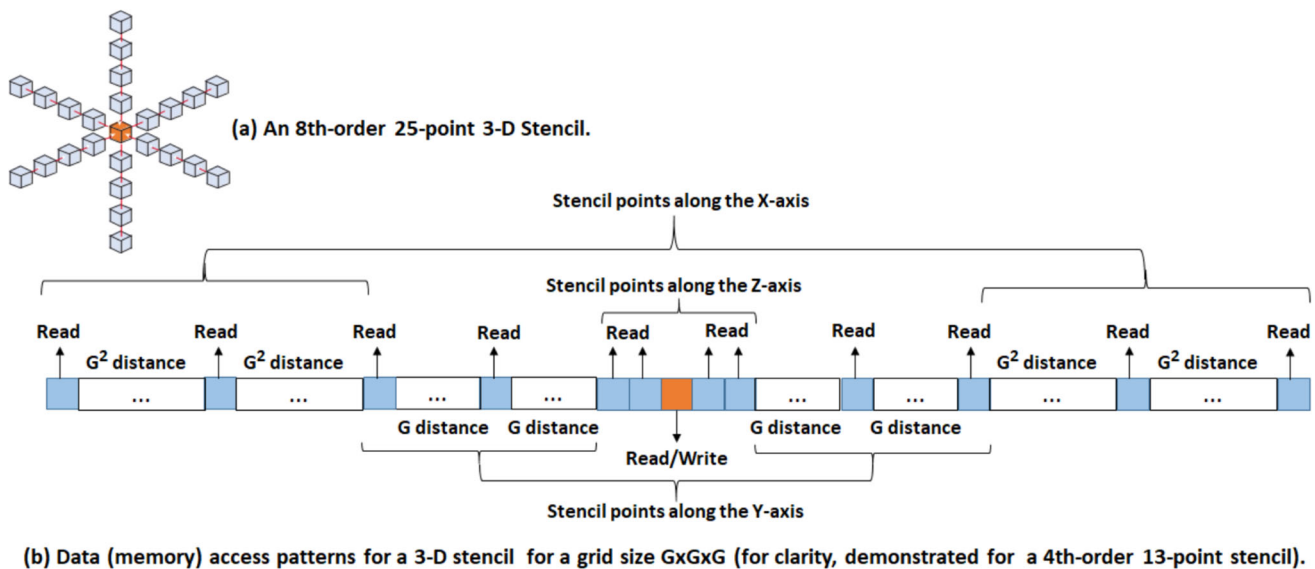


Fig. 1 a Illustration of a 25-point 8th-order Stencil (a), and b data access patterns for a 13-point 4th-order 3D stencil for a grid of size $G \times G \times G$

power efficiency. FPGAs, however, have relatively lower frequencies and smaller on-chip local memory than their CPU and GPU counterparts. Many researchers investigated different implementations of stencil kernels on FPGAs attempting to optimize their performance and eliminate bottlenecks. Two main strategies were often employed to maximize data locality and exploit parallelism; Spatial blocking and temporal blocking. In spatial blocking, large input grids stored in the FPGA board's memory are divided into smaller blocks to fit into local FPGA memory (Block RAMs). Temporal blocking refers to computing several iterations on the same block to reduce memory access. In [7], multiple iterations were combined into a single iteration by direct substitution into the stencil (i.e. replacing value with their whole stencil values from the previous iteration). This effectively increased the stencil order significantly and thus cannot be applied to higher order stencils. Many researchers [2]–[5] proposed pipelined data flow models to effectively pipeline the stencil computations for several iterations (loop unrolling) as illustrated in Fig. 2. This approach might be slower and requires more computations compared to the previous one, but it improves iteration-level parallelism and scalability.

Spatial blocking (also called tiling) creates the problem of halo regions (or ghost regions). Data at the boundary of a block cannot be computed since it requires data from other neighboring blocks. This means that after processing one iteration of a block, data at the boundaries will become invalid. The area composed of invalid data is called halo region. Combining both spatial and temporal blocking techniques enlarges the halo region. The size of halo region increases proportional to the width of temporal blocking. The direct solution of the halo problem is to have overlapping tiles, i.e.

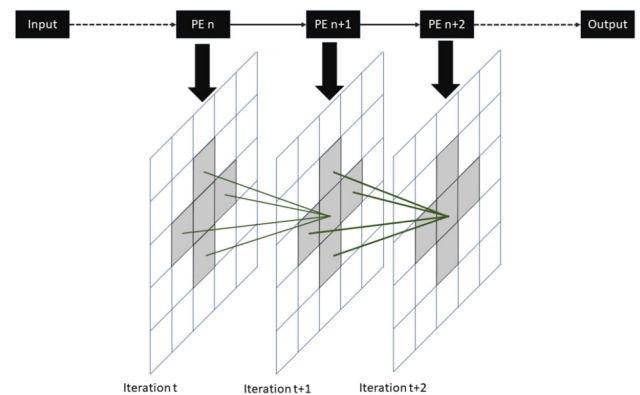


Fig. 2 Processing multiple iterations of the stencil using a pipeline of processing elements (PEs)

each tile would include the required boundary points from the previous iteration such that at the end of the number of combined/fused iterations we end up with the complete tile. This can lead to considerable duplication of data and computations as illustrated in Fig. 3.

To alleviate the effect of halo region, Wang et al. [12] proposed to communicate data at the boundaries between adjacent concurrent blocks. This approach eliminates redundant data and computations but at the cost of synchronization and communication overhead. Nate et al. [8], on the other hand, implemented a single chain of modules called streaming stencil time-step (SST) connected together in a pipelined manner. Each SST computes a single iteration of the array and forwards the output to the next SST. SST's deeply pipelined design, however only exploits temporal parallelism. The spatial parallelism was exploited in the deeply pipelined design in [2, 4, 5] by vectorizing the computation in each iteration.

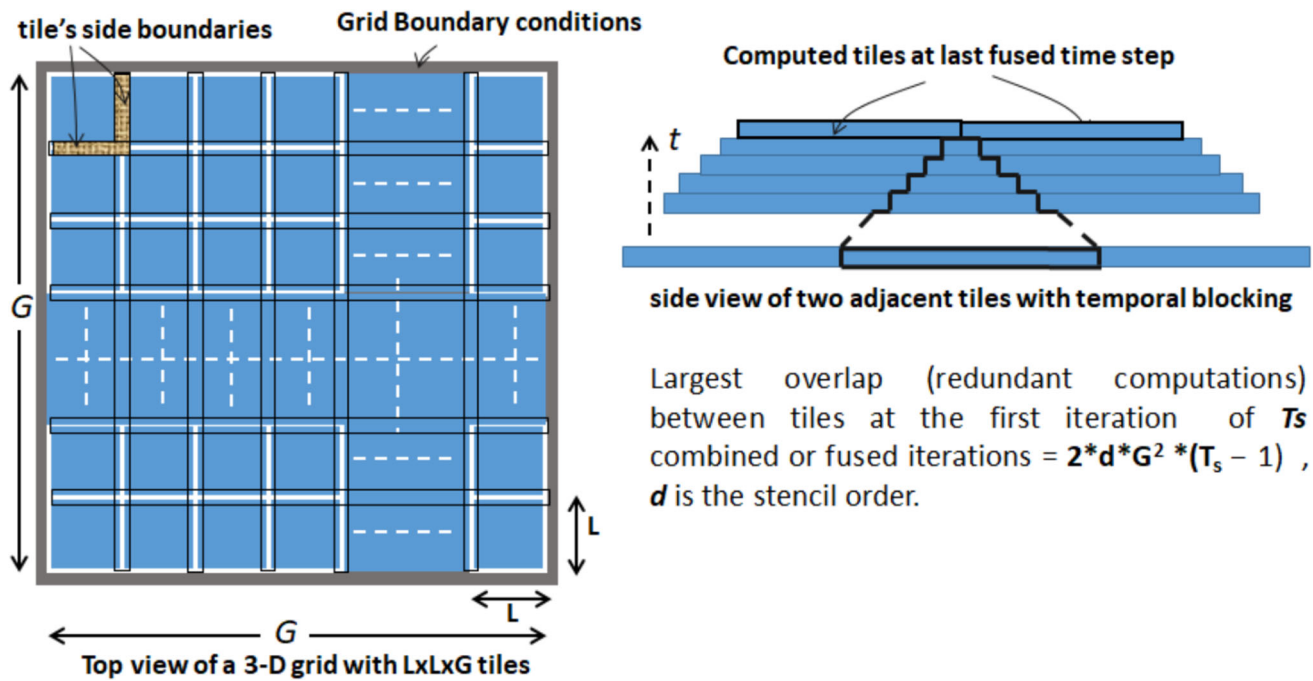


Fig. 3 Redundant point calculations due to the halo problem (illustrated on 2D tile)

This means that in each iteration a number of consecutive elements are computed concurrently. This design was extended to support multiple FPGAs in [7] and [10] to enhance parallelism in both spatial and temporal domain. In [13] further improvements were proposed based on direct computation of multiple iterations and pre-computing the stencil coefficients for these fused iterations at design time. This improved both throughput and on-FPGA memory requirements. Recently, Senju, an automation framework for parallelizing iterative stencil loops on single/multiple FPGAs based on the SST methodology was proposed in [14]. It utilizes a hybrid FIFO approach that logically group single and packed data buffers for better resource utilization. This improve the ability to support more spatial parallelism (with vectorized computations). Another framework for parallelizing iterative stencil loops based on both spatial and temporal parallelism is the SODA framework [2]. SODA generates the vectorized on-FPGA compute engines (one per iteration) that consist of data re-use buffers in the form of multiple FIFOs and stencil processing elements (PEs). The total grid size is limited by the FPGA off-chip memory (i.e. FPGA board's DDRAM) size, and the tile size is limited by the on-chip memory. Data is streamed from the FPGA off-chip memory to these FIFOs. The overall design strategy is to maximize data re-use and hence minimize off-chip memory access. This allow the accelerator to efficiently utilize the off-chip memory bandwidth. FIFOs sizes are adjusted to hold enough data points to keep the stencil computing engine(s) working at maximum rate. The FIFOs have multiple tabs to forward data in

different locations to the stencil PEs as specified by the specific stencil's access patterns. Hence, once a grid point has all its neighboring stencil points available in the buffers, the PE can compute the next iteration's value. Data points move through the FIFOs till they are no longer needed (each point is only read once from the off-chip memory and used multiple times). Additional FIFOs forward computed points to the next iteration's compute engine before the final iteration points are written back to the FPGA off-chip memory. Even though Senju reportedly achieved better energy efficiency than most of the existing frameworks, the SODA framework produced designs with higher frequencies even with FPGAs fabricated with older technology nodes. SODA also generates efficient stencil implementations as OpenCL [1] kernels based on user-specified stencils and other design parameters (like tile dimensions, unrolling factor, and number of combined or fused iterations). Both frameworks however perform tiling/untiling from the FPGA's off-chip memory. With large 3-D stencils, the memory access patterns for these operations (tiling/untiling) become highly irregular which result in huge degradation of the memory bandwidth (up to an order of magnitude). Furthermore, boundary conditions computations, which require different stencil at the boundary points, are not explicitly incorporated into these frameworks. These would degrade the off-chip memory bandwidth further and require separate on-FPGA compute engine(s) with separate FIFOs, which is not easy to incorporate into the frameworks.

Reviewing existing works on iterative stencil acceleration on FPGAs, the following can be noted:



1. Majority of works only consider 1D or 2D stencils. When 3D stencils are considered, only relatively small stencils are implemented (e.g. 5 points). Large (i.e. high-order) 3D stencils have extremely irregular data (memory) access patterns. This does not only reduce the memory bandwidth but also complicates the on-FPGA buffer design. Many of the proposed methodologies and frameworks actually become impractical at large 3D stencils (e.g. 8th-order 25-point stencils as in this work) and large grid sizes,
2. Most of the reported throughputs and powers are for the on-FPGA implementation, without considering host-FPGA communication overhead,
3. There is no mention or consideration of the need to save intermediate iteration values of the grid on the host. Many applications require saving the grid value at least every few time steps for later use,
4. Computing the grids' boundary conditions (i.e. the grid values at the boundary points) are not discussed at all. These points require values at points outside the grid. Boundary values can be computed based on one of several possible boundary conditions (e.g. Convolutional Perfectly Matched Layers for Absorbing boundary condition) that handle the decay phenomena of the grid values outside the grid. These may not only be complex, but also utilize different stencils than the main stencil. They are also only performed on outer boundaries of the whole grid, further increasing irregularity in memory accesses. Depending on the stencil size, a 3-D grid requires computing 6 multi-point wide planes of boundary values.

The above issues complicate tiling 3D grids from the FPGA off-chip memory as most researchers seem to be proposing and limits the grid size. With the limited capacity of FPGA off-chip memory, these issues, would adversely affect the overall performance significantly. In this work, an efficient methodology to accelerate computing of high-order stencils on large 3D grids on FPGAs is proposed. Unlike previous works, the proposed methodology successfully addresses all the above issues. More specifically, the contribution of this work can be summarized as follows:

1. A complete methodology for accelerating high order 3D stencil over *any* grid size utilizing both spatial and temporal blocking as a fully asynchronous pipeline. Unlike previous works such as [2] and [14], tiling and untiling (re-construction of the computed grid) of the grid is performed from the host CPU's memory and not from the FPGA's off-chip memory. Utilizing the host's larger and faster memory for tiling not only improved the resource utilization on the FPGA (allowing processing of larger tiles) but also significantly reduced the pressure

- on the FPGA off-chip memory bandwidth resulting in higher stencil computation on the FPGA. It also allowed computing the boundary conditions on the host in parallel with the on-FPGA computations, further improving FPGA resource utilization and off-chip memory utilization,
2. Ability to save intermediate grid values on the host's disk without any slowdown of the stencil computations. This was not addressed by any previous works even though it is required by many applications and can adversely affect performance,
3. A full implementation, performance evaluation and optimization of the proposed methodology on a commodity server using a 25-point 8th order stencil (for reverse time migration),
4. Performance and power efficiency comparisons with an A100 NVIDIA GPU implementation of the same stencil and previously published works of the same stencil on multi-core CPUs,
5. Extending the proposed methodology to employ two FPGAs for accelerating the same stencil and evaluating the potential increase in performance.

Detailed description of the proposed methodology including expansion to multi-FPGAs is provided in Sect. 2. Experimental results for optimizing and evaluating the different design parameters under the proposed methodology are presented in Sect. 3. Comparisons to a GPU implementation and previously published works are given in Sect. 4. Evaluation of the proposed methodology for two-FPGA systems is presented in Sect. 5 followed by conclusions in Sect. 6.

2 The Proposed Methodology

As was mentioned before, tiling (i.e. spatial blocking) of large 3D grids is essential due to the limited capacity of FPGAs' local memories (embedded SRAMs). 3D tiling leads to highly irregular memory access patterns (for both, making the 3D tiles, and combining them back into the 3D grid). The irregular accesses degrade the memory performance, and hence the whole application significantly. There are basically two possible options for tiling and untiling (i.e. combining the tiles back into the 3D grid):

1. Send the whole 3D grid to the FPGA off-chip memory, then the on-FPGA engine(s) performs tiling, moves the tiles to the on-FPGA buffer(s), compute the stencil over the tile, move the tile back to the off-chip memory and get a new tile, etc. The FPGA would also perform the untiling, and compute the boundary conditions before the next iteration can start. If the grid points need to be saved, the whole grid need to be sent back to the host's

disk every few iterations. This is the approach that all previous works seem to follow (many works do not give any details about how they do the tiling/untiling). This limits the 3D grid size to the FPGA off-chip memory size. This approach consumes many of the FPGA resources for tiling and boundary conditions computations. It also increases the pressure on the FPGA's off-chip memory bandwidth. For the FPGA board being used, the total bandwidth of the FPGA's off-chip memory is ~ 80 GB/s (4 channels DDR4) with 2 channels reserved for host-FPGA communications (i.e. ~ 40 GB/s). This leaves only two channels or ~ 40 GB/s maximum bandwidth for the on-FPGA circuitry. This remaining bandwidth can only be attained with bursty and regular memory access (i.e. accessing data along the storage access order, in this case the Z-axis). If tiling/untiling is performed on the FPGA, the irregular memory access causes the effective memory bandwidth to be degraded by an order of magnitude (~ 4–7 GB/s in our case). It should be noted that this degradation is more prominent because of the 3-D nature of the tiles.

2. The approach developed for this work performs tiling on the host's CPU, send the individual tiles to the FPGA to perform the stencil computation then receive the computed tiles and reconstruct (un-tilde) the grid, compute the boundary conditions, save the grid to the disk, and repeat. This approach leverages the host CPU's superior memory capacity and throughput (even with irregular memory access patterns, the sustained average memory throughput on the host is more than ~ 40 GB/s). Hence, it allows for complete overlap of tiling/untiling, CPU-FPGA communication, and on-FPGA stencil computations. Additionally, it utilizes the CPU's computing power to handle boundary conditions computations.

2.1 The Asynchronous Execution Pipeline

A fully asynchronous 5-stages pipeline is implemented under the proposed methodology. The 5 stages working in parallel are: a tiling stage and a sending stage, both on the host CPU, an on-FPGA processing (stencil-computing) stage, a receiving stage and an untiling (grid reconstruction and boundary conditions computations) stage, again on the host CPU. The tiles received from the FPGA are either internal tiles or boundary tiles. The latter are used to compute the boundary conditions before they are written back to the complete grid (i.e. before they are *untiled*). Figure 4 illustrates how the proposed 5-stage pipeline works. All pipeline stages except for the on-FPGA processing are managed by host SW threads.

The stages of the asynchronous pipeline were optimized to balance the pipeline stages' latencies and achieve maximum throughput. As will be shown later, several highly-correlated

design parameters had to be simultaneously optimized. These included the tile size, which directly affects all stages' latencies and throughputs, and the number of fused time steps, which affects tiles overlapping and the size boundary conditions computations (more fused steps result in larger boundary regions).

The 8th order 25-point 3D floating point stencil used in this work is shown below:

$$P_{i,j,k}^{t+1} = O_{ii}^8(P_{i,j,k}^t) + O_{jj}^8(P_{i,j,k}^t) + O_{kk}^8(P_{i,j,k}^t).$$

where $O_{ii}^8(P_{i,j,k}^t)$ represents the 8th-order approximation of the second partial derivative at the point $P_{i,j,k}^t$ along axis i , defined as:

$$\begin{aligned} O_{ii}^8(P_{i,j,k}^t) = & -\frac{205}{72}P_{i,j,k}^t \\ & + \frac{8}{5}(P_{i+1,j,k}^t + P_{i-1,j,k}^t) \\ & - \frac{1}{5}(P_{i+2,j,k}^t + P_{i-2,j,k}^t) \\ & + \frac{8}{315}(P_{i+3,j,k}^t + P_{i-3,j,k}^t) \\ & - \frac{1}{560}(P_{i+4,j,k}^t + P_{i-4,j,k}^t). \end{aligned}$$

The same goes for $O_{jj}^8(P_{i,j,k}^t)$ and $O_{kk}^8(P_{i,j,k}^t)$ along axes j and k , respectively.

This stencil is widely used to compute 2nd-order partial derivative in 3D spaces. 3D arrays are usually stored in the Z-dimension (the *fast dimension*) first, then the Y-dimension (the *intermediate dimension*), then the X-dimension (the *slow dimension*). The proposed methodology partitions the application into two partitions: the on-FPGA *stencil kernel* and *host application*.

2.2 Stencil Kernel

The stencil kernel used in this work is mostly based on the SODA automation framework [2]. Figure 5 shows the underlying microarchitecture of SODA-generated kernels. It is composed of multiple pipelined stages. Each stage consists of multiple First-In-First-Out (FIFO) and Flip Flop (FF) units that feed the appropriate points to the PEs of the stage at the appropriate time. SODA defines three design parameters: input size, unroll factor, and iterate factor. It also requires all dimensions of the input grid to be pre-determined except for the slow dimension which is determined at run time as data are streamed along this dimension (as x–y planes). The unroll factor determines the number of PEs in each stage. The iterate factor indicates how many iterations are implemented in the design (the number of compute engines). It should be noted



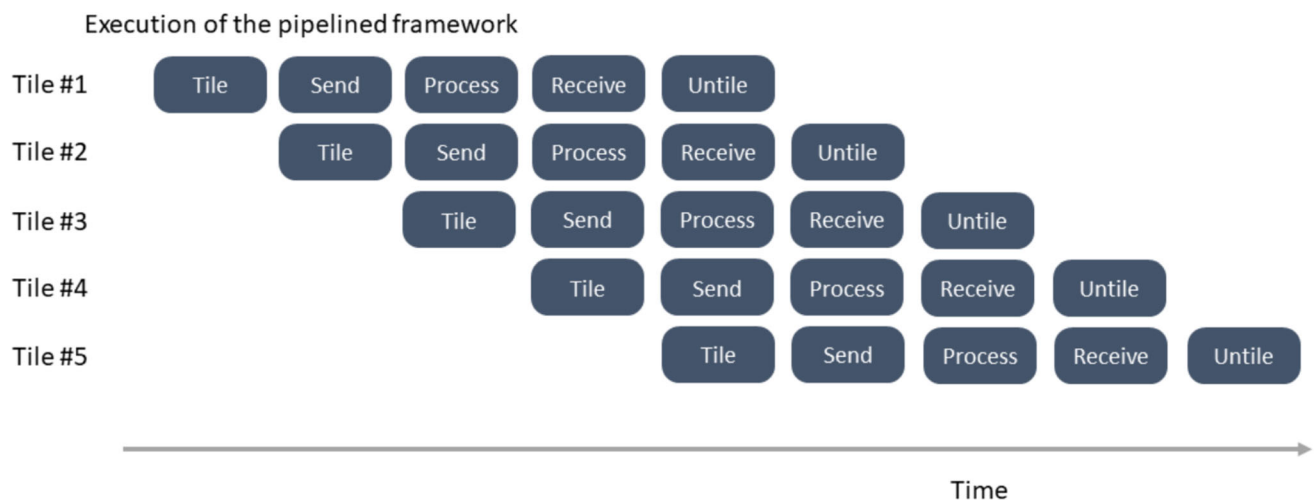


Fig. 4 The proposed 5-stage execution pipeline

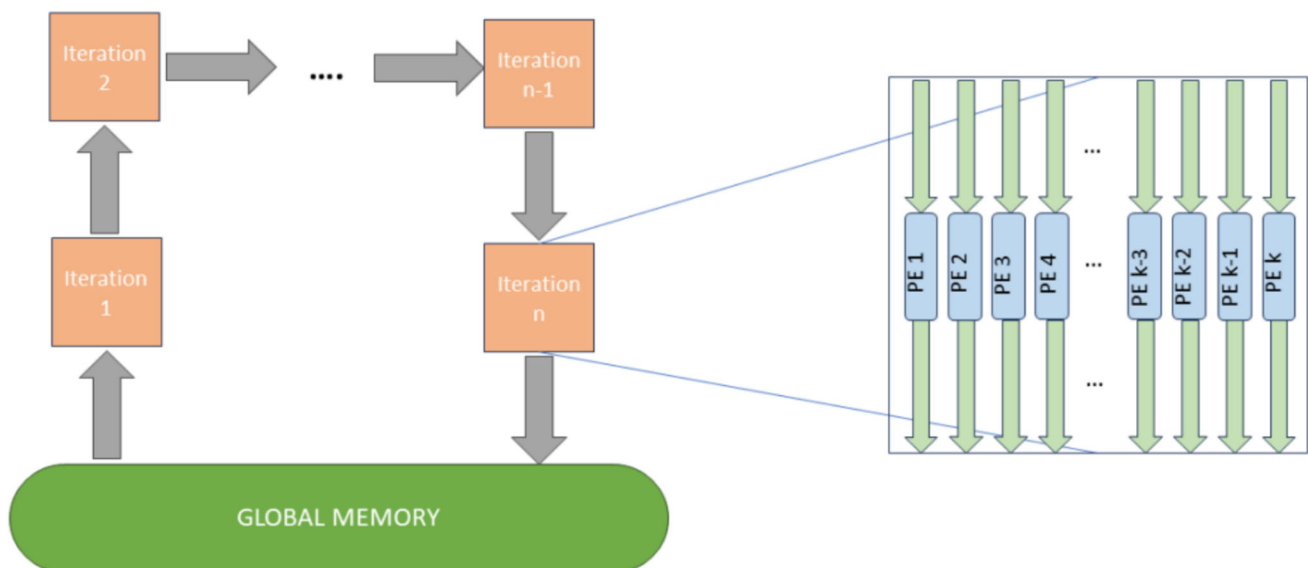


Fig. 5 General microarchitecture of SODA-generated kernels

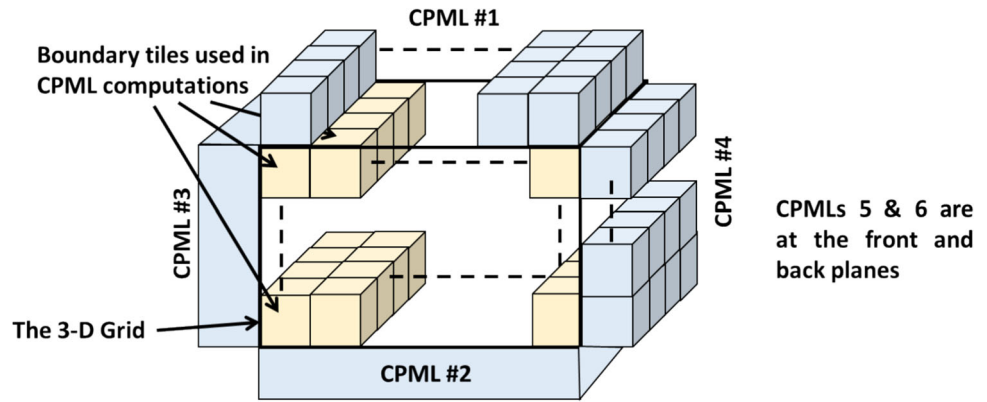
that the SODA framework was only used to generate the on-FPGA circuitry for a specified tile size. The other stages of the asynchronous pipeline (tiling, boundary conditions computations, and untiling) and coordination of all stages were all implemented and optimized under the proposed framework/methodology.

2.3 Computing Boundary Conditions

As was mentioned before, boundary conditions are computed by the *untiling* stage as part of re-constructing the grid before the next time step starts. For this work the Convolutional Perfectly Matched Layer (CPML) approach was used to handle the grid boundaries. This approach is highly effective for various applications where the grid represents a wave field

like seismic, elastic, and electromagnetic waves. CPML provides a way to compute the grid's boundary values and absorb outgoing waves so they do not reflect due to the grid's discontinuity. For 3-D grids, it utilizes six non-physical CPMLs that are added to the six edge planes of the 3-D grid as illustrated in Fig. 6. Each CPML is 10 points wide. Waves entering these layers are gradually attenuated, or absorbed, without causing reflections. CPML handles waves behavior more effectively than traditional methods, particularly for waves that hit the boundary at oblique angles or for the more complex "second-order" wave equations. The CPML equations along axis i are defined as follows:

Fig. 6 Illustration of the six Convolutional Perfectly Matched Layers (CPMLs) used to compute the absorbing boundary conditions. The figure also shows the 3-D tiles at the boundaries that are involved in the computation. The CMLs are 10 points wide added outside the grid for the wave decay



1. 1st, the base damping coefficient $d0$ is computed as follows:

$$d0 = -(cpml_pow + 1) * cpml_vmax \\ * \log [rcoef] / (2 * m_nlayer * m_dl)$$

where $cpml_pow$ is an exponent controlling the damping profile shape, $cpml_vmax$ is a constant related to wave speed (maximum velocity), $rcoef$ is the reflection coefficient, m_nlayer thickness of CPML layers, and m_dl is the grid spacing. The above formula ensures that the damping increases smoothly across the CPML layers to minimize reflections.

2. Then the normalized depth of the current grid point (with grid index iz) within the CPML with respect to the starting index to the CPML region m_Si1 in the direction of the CPML layer (e.g. CPML number 1 is in the Y direction):

$$dnorm = (m_Si1 \\ - iz) / (m_nlayer) \text{ (if } dnorm < 0, \text{ it is clamped to 0)}$$

3. Then compute the damping term α : $\alpha = cpml_alpha_max * (1 - dnorm)$ ($cpml_alpha_max$ is a user set scaling factor)

4. Then the spatial damping profile dd is computed: $dd = d0 * dnorm^{cpml_pow}$

5. Then the CPML coefficients are updated setting up the exponential decay ($bpml$) and auxiliary coefficient ($apml$) for the CPML update equations at CPML grid points (indices: $ipmlx$, $ipmly$, $ipmlz$) for the next time step (dt is time step):

$$bpml[ipmlx][ipmly][ipmlz] \\ = \exp(-(dd + \alpha) * dt) \\ apml[ipmlx][ipmly][ipmlz] \\ = dd * (bpml[ipmlx][ipmly][ipmlz] - 1) / (dd + \alpha)$$

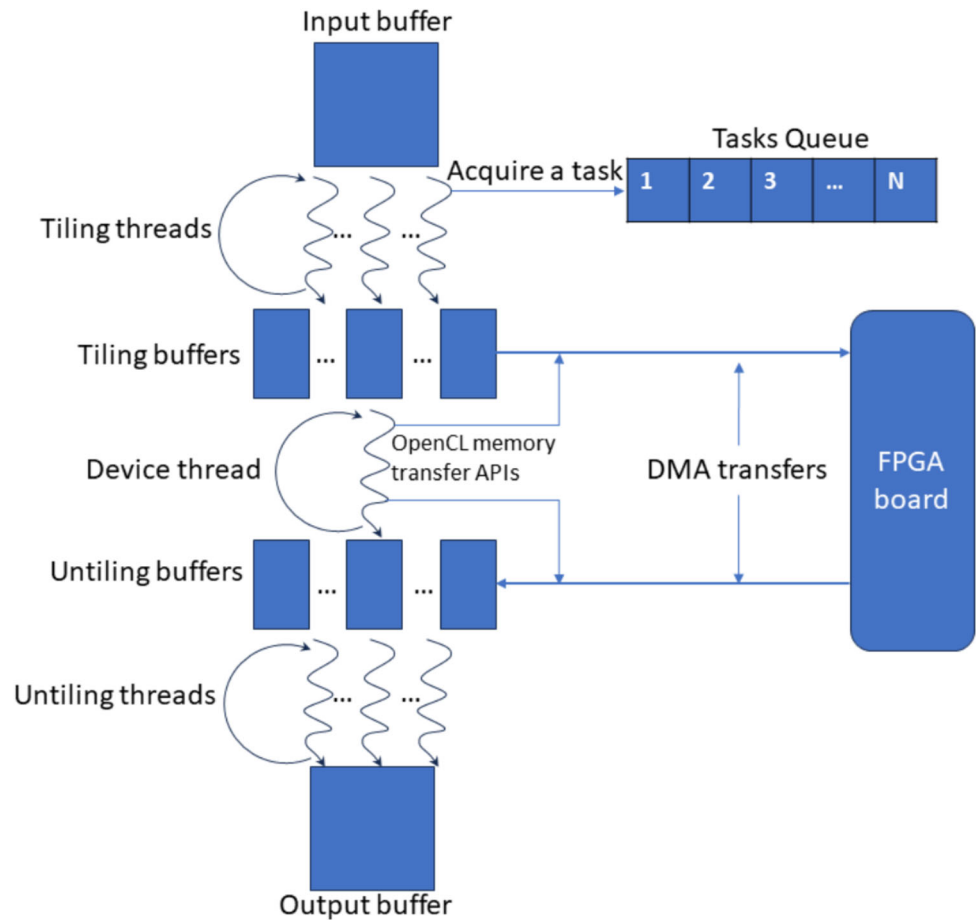
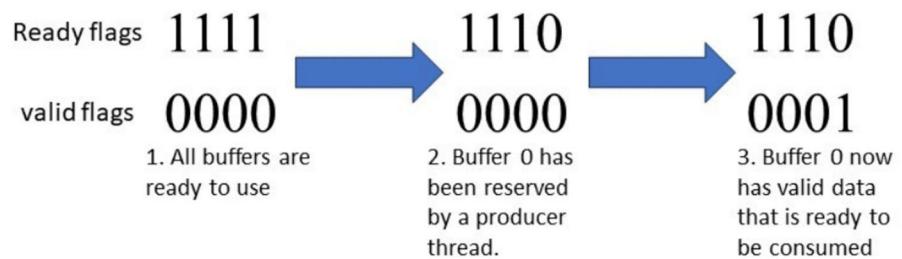
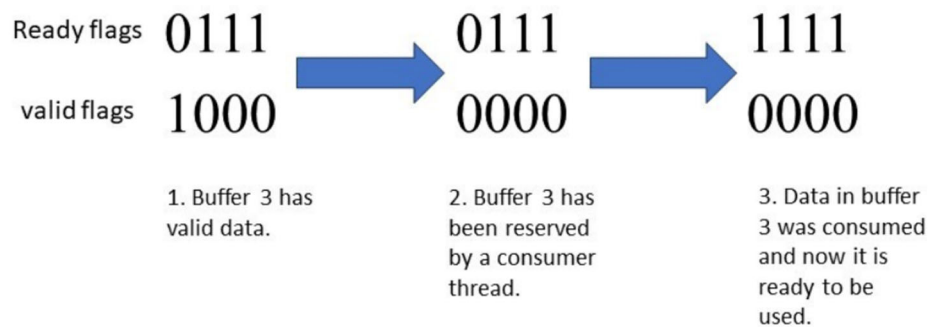
Boundary grid tiles, as illustrated in Fig. 6, are used to compute the CPML layers and update the grid's boundary values as soon as they are received by the *untiling* stage.

2.4 Host Application

A special host application was developed to implement the asynchronous pipeline and orchestrate the operation of the different stages. Using OpenCL APIs to communicate with FPGAs, the host application spawns a thread called *device thread* for each FPGA board attached to the host. A *device thread* is responsible for tiling and untiling the stencil grid (or sub-grid in case of multiple FPGA boards) and communicating with the FPGA. Each *device thread* spawns multiple *tiling* and *untiling* threads that are assigned (i.e. pinned) to different physical cores within the same host socket to maximize their performance. Each *device thread* starts its execution by calculating the number of tiles (based on the input grid size and specified tile size) and their addresses (in host's RAM) in sub-grid memory buffer assigned to it, and then fills a task queue with tiles' addresses. The *tiling* threads then start grabbing tiling tasks from the queue, perform tiling of the specific block (i.e. copying it to a RAM buffer), and then signal the *device thread* with the location and the index of the tile. The *device thread*, then issues non-blocking commands (API) for sending the tile to the FPGA where it is processed, and then receives the results back. Once a tile is received back, the *device thread* signals the *untiling threads* with the index and location of the tile. *Untiling threads* finally un-tiling the resultant tiles to the output buffer that holds the whole grid. Figure 7 shows the execution model of single device thread.

To minimize the latency of tiling and untiling processes and ensure that they do not become a bottleneck (especially the untiling process since it includes boundary conditions computations), two strategies were adopted. These processes were overlapped with memory transfers to/from the



Fig. 7 Execution model of a device thread**A. Producing data****B. Consuming data****Fig. 8** Illustrating the multiple buffering technique with 4 common buffers

FPGA using multiple (common) buffers combined with simple asynchronous signaling to avoid hard synchronization delays (that are incurred by thread barriers or data locks). Each buffer has two 1-bit associated flags: *ready* and *valid*. *Ready* flags denote buffers that are ready to be used by *producer* threads (e.g. tiling or receiving computed tiles from the FPGA) while *valid* flags denote buffers that have valid data for *consumer* threads (e.g. untiling or sending threads). Whenever a producer needs to produce data, it scans the buffers in order to look for an unused buffer (with a *ready* flag = 1), resets its *ready* flag and proceeds to writing to that buffer. As soon as it finishes, it sets the buffer's *valid* flag. Consumers, on the other hand, scan buffers (in order) for anyone flagged as *valid*. Once a valid buffer is found, its valid flag is reset, and its data is consumed. When a consumer is finished moving the buffer's data, it sets its *ready* flag. Figure 8 illustrates these processes with 4 buffers. The number of *producer* and *consumer* threads equals the number of tiles.

2.4.1 Tiles Overlapping

In order to overcome the Halo region problem, the 3D grid is divided into overlapping tiles (only along the Z and Y directions as tiles are streamed in the X dimension). The amount of overlap points that should be added around a tile depends on the stencil length (d), the total grid (domain) size (G), and the number of combined or fused iterations (T_s). Assuming a 3D grid of size G (i.e. $G \times G \times G$) divided into equal 3D tiles of dimensions $L \times L \times G$, then $n = G/L$ is the number of tiles along the Z or Y dimensions. Figure 3 illustrates the overlap regions between adjacent tiles due to fusing several iterations. The total number of redundant points computed due to overlapping between adjacent tiles (note that the overlapped regions are computed twice and that there are no overlap regions along outer grid boundaries):

$$\text{Redundant Points} = 4(n-1)G^2 \sum_{i=1}^{T_s-1} (i*d)$$

The total number of grid points (including outer boundaries) can be computed as follows:

$$T_s * G^3 + 6 * G^2 \sum_{i=1}^{T_s-1} (i*d)$$

For an 8th-order 25-point 3D stencil ($d = 4$), and a grid size $G = 1024$, a tile size $L = 256$, and 5 fused iterations (i.e. $T_s = 5$), the redundant points represent ~ 15% of the total computed points. Even though this is a significant reduction in the effective throughput, it is still better than the alternative which would require storing the computed (east and south)

boundaries of each tile on the FPGA local memory or board RAM to be used by subsequent tiles. Just to illustrate the amount of buffering that would be required to store computed tiles' boundaries, let us assume that tiling starts from the north-west end of the grid and proceed row-wise eastward. The computed east boundary of a tile will become the west boundary for the next tile and the south boundaries of a row of tiles will become the north boundaries for the next row's tiles. Saving these boundaries would effectively reduce the redundant point calculations by 50% (or ~ 8% of the total computed points for the above example). Only one buffer will be required for the east boundary (to be overwritten by each subsequent tile), but n buffers will be required for the south boundaries. Thus, the total required buffer size would be:

$$\text{Buffer Size} = 4(n+1)*L*G* \sum_{i=1}^{T_s-1} (i*2d)$$

Using the same parameters as in the above example, the required buffer size would be ~ 400 MB which cannot fit on the FPGA's local memory. Storing these boundaries on the FPGA off-chip memory (global memory) would adversely impact the main stencil kernel's speed as it would compete for global memory bandwidth with the circuits that read the overlap buffers (each iteration's compute engine would be reading the relevant boundary data while the 1st iteration's engine is reading the tile's data).

2.4.2 Extending the Methodology to Multiple FPGA Devices

The developed methodology is extended to support two host sockets each with an attached FPGA device, all working in parallel. Multiple FPGAs attached to the same socket won't provide extra speed unless there are direct connections between the FPGAs, something that we could not explore with the commodity server used in this work. In the case of a single FPGA, the *device thread* loads the whole input grid into its input buffer and starts processing it. To avoid excessive synchronization between the two FPGAs circuitries, the logical solution was to divide the 3-D grid equally among the two sockets/FPGAs as illustrated in Fig. 9. Boundary points in each half along the cut-plane boundary, however need to be exchanged among the two FPGAs as they are needed by boundary tiles on the other half. The amount of boundary points that need to be exchanged depend on the stencil size (d), grid size ($G \times G \times G$), and the number of fused time steps (T_s) as shown below:

$$\text{No. of Exchanged Points} = G^2 * d * T_s$$

The host application loads one sub-grid into each socket's input buffer. At the beginning of each step, data at boundary



Fig. 9 Boundary exchange between the two sub-grids: boundary tiles are shown in yellow color, while the exchanged boundary points are shaded in brown

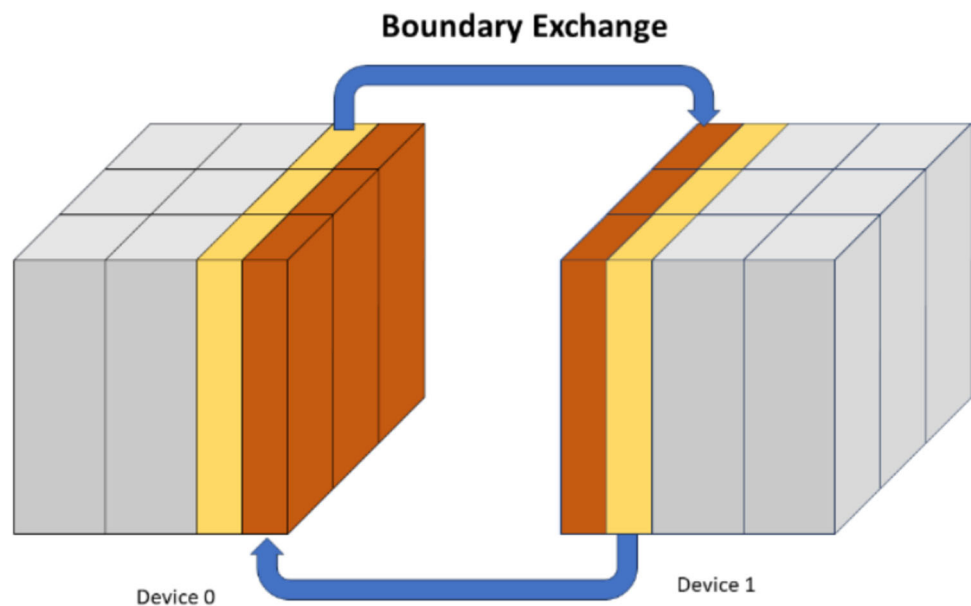


Table 1 Specification of the host machine

Part	Model
CPU	Dual Sockets Intel E5-2620 @ 2.40 GHz (12 Physical Cores, 24 Logical Cores/Threads)
RAM	512 GB DDR4 (16 × 32 GB DIMMs)
HDD	4 × 2 TB in RAID 10 (RAID0 + 1) Configuration
OS	CentOS Linux 7

of both sub-grids are exchanged by a dedicated thread. This thread works simultaneously with devices threads. During data exchange, tiles at the boundaries of both devices are locked with a barrier to avoid processing invalid data. Computations of other tiles proceed as usual. Since the boundary exchange time is much smaller than the processing time of the other tiles, its delay is completely masked. However, it does affect other delays via memory contention and cache pollution. Figure 9 shows the boundaries exchange process.

3 Experimental Results

3.1 Experimental Setup

Experiments were performed on a host machine with the specifications shown in Table 1. Stencil kernels were implemented on an Intel's Stratix 10 SX FPGA (fabricated in a 14 nm technology) development board attached to the host via 16-lane PCIe Gen3. The board has 8 GB, 4-channel DDR4 memory running at 1333 MHz (i.e. 2666 Mega transfers per second). OpenCL dedicates two channels for host-FPGA

communications leaving two-channels for the on-FPGA circuitry with a peak bandwidth of 40 GB/s.

To ensure the generality of the results, the following points have been taken into account:

1. The FPGA used in this work, Stratix 10 SX, utilizes the same re-configurable logic (including standard DSP blocks), interconnects, and embedded RAM resources typically found in any modern FPGA made by the same vendor or other vendors. Hence, the proposed framework can be synthesized on any other FPGA. As mentioned in the Introduction section, the tile size is limited by the total size of on-FPGA embedded RAM. As will be shown next, the proposed framework has been evaluated using several tile sizes and parallelization factors further ensuring the applicability of the results and conclusions to other FPGAs with different RAM sizes and/or number of configurable logic resources,
2. Following up on point 1 above, no special feature or component on the Stratix 10 SX was utilized to implement the proposed acceleration framework, further ensuring general applicability of the proposed framework,
3. Finally, OpenCL, a standard framework for heterogeneous computing, was utilized for synthesizing the on-FPGA circuitry, and the communication and synchronization with the host application.

3.2 Optimization of Stencil Kernel Throughput on FPGA

Extensive experiments were carried out to quantify the performance of the SODA-generated HW kernels on the FPGA

Table 2 Throughput in (GB/s) versus Unrolling factors (UF) and number of combined iterations (Ts) for three grid slice sizes (3rd grid dimension kept at 1024 points)

UFTs	1	2	3	4	5
Slice size: 128×128					
1	2.00	3.93	5.93	8.00	9.40
2	4.00	8.00	12.00	15.99	19.99
4	7.99	15.99	23.98	30.27	39.98
Slice size: 256×256					
1	1.83	3.41	5.36	7.33	9.40
2	3.78	6.86	10.96	14.93	18.13
4	7.93	15.99	22.98	31.98	35.71
Slice size: 512×512					
1	1.49	2.45	–	–	–
2	3.14	6.66	–	–	–
4	7.26	13.50	–	–	–

Table 3 On-FPGA Block RAMs (M20K Blocks) usage versus UF and Ts for three grid slice sizes (3rd grid dimension kept at 1,024 points)

UFTs	1	2	3	4	5
Slice size: 128×128					
1	550	997	1444	1891	2338
2	680	1256	1832	2408	2984
4	922	1738	2554	3370	4186
Slice size: 256×256					
1	1318	2533	3748	4963	6178
2	1448	2792	4136	5480	6824
4	1722	3338	4954	6570	8186
Slice size: 512×512					
1	4390	8677	–	–	–
2	4520	8936	–	–	–
4	4794	9482	–	–	–

by exhaustively varying the FPGA design parameters (3D input grid size, number of combined (or fused) iterations, and the unrolling factor) under several constraints. One set of constraints relates to the available FPGA resources (size of on/off chip memories, and number of other FPGA resources such as LUTs, DSP blocks, and routing resources). Any design point that could not fit on the FPGA was excluded (e.g. Ts values > 2 when the grid slice is 512×512). Another set of constraints relates to the SW threads running on the host CPU. Specifically, any parameters combination that made the tilling/untilling stages of the asynchronous pipeline slower than the FPGA stages were also excluded (e.g. the maximum value of Ts was limited to 5 as the number of boundary points that need to be computed in the untilling stage becomes excessive and leads to that stage being much slower than the rest). The values of the three parameters were varied as follows:

1. Since SODA implementation streams the grid along the X axis (slow dimension), only the size of the 2D slice (i.e.

$G \times G$) of the HW grid was tuned keeping the slow (i.e. X) dimension at 1024 points as experiments had shown that this dimension (where slices are streamed across) does not have a noticeable impact on the effective throughput. Three different grid's Z-Y slice sizes were explored (all are powers of 2 for convenience): 128×128 , 256×256 , and 512×512 . Larger slice size could not be fitted on a single FPGA.

2. The number of combined iterations, Ts, varied from 1 to 5. The maximum value of Ts was limited to 5 as the number of the boundary points beyond that value makes the untilling stage slower than the other pipeline stages.
3. There values were used for the unrolling factor, UF, which represents the number of points computed in parallel in each iteration (or the number of PEs). The unroll factors that were used are 1, 2, and 4.

With an input grid's slice size of 512×512 , only two iterations could fit in the FPGA as we ran out of embedded RAM



Table 6 Breakdown of power dissipation components of one of the kernels (grid size of $128 \times 128 \times 1024$, UF = 4, and Ts = 5)

Total Thermal Power Dissipation = 39,467.22 mW	
Total Transceiver Thermal Power Dissipation = 7476.53 mW (18.9%)	
Transceiver Standby Thermal Power Dissipation	3164.15 mW
Transceiver Dynamic Thermal Power Dissipation	4312.38 mW
Total I/O Thermal Power Dissipation = 11,934.75 mW (30.2%)	
I/O Standby Thermal Power Dissipation	1787.97 mW
I/O Dynamic Thermal Power Dissipation	10,146.78 mW
Device Static Thermal Power Dissipation = 6653.19 mW (16.8%)	
Core Dynamic Thermal Power Dissipation = 13,485.72 mW (34.1%)	

blocks. So, a total of 36 kernels were generated and evaluated. Since every Ts iterations are combined, throughput in (GB/s) was measured as follows:

$$\text{Throughput} = \frac{\text{Ts*size of the grid in GB}}{\text{Total processing time in s}} \text{ GB/s}$$

Table 2 summarizes the measured throughput of all implemented kernels. As expected, throughput scales perfectly with both UF and Ts, up to the global memory bandwidth (~ 40 GB/s). The measured throughput at grid's slice size of 128×128 was slightly better than that at 256×256 slice size with Ts = 5 (~ 40 GB/s versus 36 GB/s). However, the percentage of Halo points (that would be computed twice when tiling is implemented) decreases linearly with the grid's slice size. So, at a 128×128 slice size and Ts = 5, Halo points represent 30% of the computed points thus effectively reducing the throughput by ~ 30% to ~ 28 GB/s). While for the 256×256 slice size, the Halo points represent 15% of the computed points, yielding an effective throughput of 31 GB/s. Hence a slice size of 256×256 is actually better. The grid size along the slow dimension X is only limited by the off-chip memory size.

Tables 3 and 4 show the kernels' block RAMs (BRAMs) and registers usage, respectively. BRAM utilization increases linearly with Ts due to the replicated re-use buffers for each iteration's PE. UF further increases BRAM utilization due to the need to store extra UF planes of data along the X direction (versus 9 when UF = 1). Similar trends are observed for registers utilization which are mostly used in the forwarding FIFOs between iteration engines. Other resources (logic or DSP blocks) have much less utilization and thus do not limit the grid slice size, Ts, or UF.

Power consumption of each kernel was also measured using the Power Analyzer of Intel's Quartus Prime Pro [6] at the kernel operating frequency (for all experiments, depending on the kernel, the frequency range was from 460 to 500 MHz) and an ambient temperature of 27 °C. Table 5 summarizes the average power consumption for two grid slice sizes at minimum and maximum UF and Ts values. These results show that power consumption is fairly independent of UF and has small dependencies on the grid slice size and Ts. As Table 6 shows, power consumption is dominated by the high-speed PCIe and DDR4 transceivers (18.9 and 30.2%, respectively), and the FPGA's standby (static) power (16.8%). The actual circuit's dynamic power is 34.1% of the total power. As such, the grid size and Ts do not impact power significantly (quadrupling the grid size increased the power by only ~ 8%, while increasing Ts by $5 \times$ (from 1 to 5) only increased the power by 38% at the larger grid size). Table 7 shows the power consumption normalized to the effective throughput for two grid slice sizes at min/max values of UF and Ts. As this table shows, power efficiency increases with UF, Ts, and grid slice size (at high Ts).

Hence, in summary, the on-FPGA kernels' throughput is slightly impacted by the grid slice size. So, to reduce the effect of Halo points (that causes more redundant calculations), the grid slice size should be the maximum that can fit on the FPGA. In addition, throughput scales perfectly with UF and Ts. Increasing Ts, however, increases the percentage of redundant boundary calculations and is also limited by availability of FPGA resources. Increasing UF allows the forwarding FIFOs to move the data faster, which requires more global memory bandwidth. Hence, increasing UF beyond the FPGA's board memory bandwidth does not increase throughput any further. In our case this meant a maximum UF of 4. Also, FPGA's resource utilization is fairly independent of UF but increases linearly with Ts. Hence, the maximum grid slice size that can be fit with a Ts of 5 was 256×256 . Finally, power consumption increases slightly with Ts as over 40% of the power is consumed in the interfaces (PCIe and DDR4 RAM). Power efficiency, however, increases with UF, Ts, and grid size.

From the above results, the optimum computing conditions for the utilized FPGA board were deemed to be grid slice size of 256×256 , UF of 4, and Ts of 5. These would yield an effective throughput of 31 GB/s (taking Halo effect into account) and 1.32 W/GB/s power efficiency.

3.3 Performance Evaluation of the Complete Pipeline

As with any FPGA-accelerated application, the main objective is to minimize latency. The average latency per iteration was measured as the sum of the pipeline stages latencies. Latencies of the pipeline stage that are managed by host SW

Table 4 Registers usage versus UF and Ts for three grid slice sizes (3rd grid dimension kept at 1,024 points)

UFTs	1	2	3	4	5
Tile size: 128 × 128					
1	71,717	125,650	180,361	235,093	289,890
2	112,980	207,334	301,645	395,848	490,236
4	197,415	376,158	553,698	731,865	910,545
Tile size: 256 × 256					
1	77,751	138,147	200,322	261,933	322,069
2	119,232	221,481	320,370	420,341	522,946
4	205,402	387,968	573,394	755,137	942,573
Tile size: 512 × 512					
1	113,107	209,676	–	–	–
2	153,823	285,837	–	–	–
4	230,314	453,784	–	–	–

Table 5 Average power consumption in Watts for two grid slice sizes

Slice size: 128 × 128			Slice size: 256 × 256		
UFTs	1	5	UFTs	1	5
1	33.07	36.44	1	35.27	46.56
4	33.91	39.47	4	35.4	48.72

Table 7 Normalized power per GB of effective throughput (W/GB/s) versus UF and Ts for two slice sizes (3rd grid dimension kept at 1024 points)

Slice size: 128 × 128			Slice size: 256 × 256		
UFTs	1	5	UFTs	1	5
1	17.06	5.5	1	17.92	5.08
4	4.39	1.41	4	4.93	1.32

Effective throughput represents the actual throughput factoring in redundant points computations

threads were measured using time stamps in the threads and averaged over all iterations. Similarly, sending, receiving, and the on-FPGA latencies were measured using OpenCL time stamps associated with queued events. A grid size of $1024 \times 1024 \times 1024$ was used with a UF of 4 for 3 tile sizes while Ts was varied from 1 to 5. The X-dimension was fixed to 1,024 for all tile sizes. The number of tiles (hence the total latency) varied with the tile size and Ts (the latter affects the

number of redundant points). Since the 5 pipeline stages do not have identical latencies, effective throughput (in GLUPs) was estimated using the average latency per pipeline stage as (the grid size is fixed at 1 G points):

$$\begin{aligned} \text{Effective Throughput} &= \frac{\text{size of the grid in points}}{(\text{total average latency per iteration}/5)} \\ &= \frac{5}{\text{total average latency per iteration}} \text{GLUPs/s} \end{aligned}$$

Table 8 Average latency (L) in milliseconds per iteration and effective throughput (ET) in GLUPs/S of the whole 3D grid (from the start of tiling the first tile, till the untiling of the last tile) versus tile size and Ts

Tile Size/Ts	1	2	3	4	5
128 × 128	L = 1340 ET = 3.73	L = 815 ET = 6.13	L = 540 ET = 9.26	L = 490 ET = 10.2	L = 460 ET = 10.87
256 × 256	L = 1680 ET = 2.98	L = 865 ET = 5.78	L = 570 ET = 8.77	L = 435 ET = 11.49	L = 356 ET = 14.04
512 × 512	L = 2750 ET = 1.82	L = 1370 ET = 3.65	–	–	–

All tiles are 1024 points in X-direction



Table 9 Breakdown of the asynchronous pipeline latencies per tile in milliseconds as a function of the tile size and Ts

Ts	Tile	Send	Execute	Receive	Un-tile
Tile size: 128×128					
1	14.7	7.8	8.9	7.3	8.2
2	12.8	7.8	8.9	7.5	7.0
3	15.1	7.7	8.8	7.4	7.3
4	13.4	7.8	9.1	7.4	5.3
5	14.4	7.6	8.7	7.3	5.5
Tile size: 256×256					
1	37.7	31.4	35.6	30.1	30.3
2	38.8	30.9	35.5	31.1	30.8
3	42.0	31.1	36.1	30.3	31.1
4	43.0	31.0	34.8	29.7	29.3
5	48.4	30.1	38.2	29.4	30.7
Tile size: 512×512					
1	88.9	124.1	151.2	155.2	88.5
2	82.6	119.5	160.6	150.1	86.9
3	—	—	—	—	—
4	—	—	—	—	—
5	—	—	—	—	—

Results were averaged over 2400 iterations of the stencil over the 3D grid (1024^3 points)

Table 8 summarizes the obtained results for the average total latency per iteration and effective throughput as a function of the tile size and Ts. As expected, the $256 \times 256 \times 1024$ tile size yielded the best latency at higher values of Ts. Table 9 shows the average latency per pipeline stage (as in Fig. 4). As expected, all stages' latencies vary with the tile size with little dependency on Ts. Also, the latency of the on-FPGA stencil computation (i.e. the execution stage) remained fairly independent of Ts. This is because the iterations PEs form a pipeline that overlaps the computations of all iterations. Irrespective of Ts, the total streaming time of a grid slice's points (from/to Off-chip memory to/from FPGA) is almost constant (there is a small variation due to the synthesized circuits' frequencies (for all experiments, the range was from 460 to 500 MHz).

4 Performance Comparisons

4.1 Comparison with GPU Implementation

The performance of an implementation on a 14 nm technology FPGA based on the proposed methodology was compared to an NVIDIA 7 nm technology A100 GPU implementation. Even though the difference in the fabrication technology nodes of the FPGA and GPU (two nodes apart) is significant, this comparison is important from power efficiency point of view. The GPU has 6912 CUDA cores and

40 GB High-bandwidth Memory (HBM2) RAM with a total bandwidth of ~ 1.5 TB/s. It was installed in a commodity server (using a 16 lane PCIe Gen 3.0 slot). All stencil computations, including the perfectly matched layers boundary conditions were computed on the GPU for 2,400 iterations. Following the single-pass approach described in [15] for implementing an 8th-order stencil, the grid is divided into 2D tiles in the fastest dimensions (Z and Y) and assigned to 2D thread blocks with shared memory. Memory management follows the approach in [16] and Nvidia's CUB library v2.8 [17] (one of CUDA C++ Core Compute Libraries) was used for thread management. Each thread iterates in the slow dimension and the grid is also streamed from the global memory in the slow dimension. This maximizes data re-use and reduce redundant computations and results. NVTOP [18] was used for performance analysis and power measurement. As was mentioned before, applications usually need the computed 3D grid values to be saved at some intervals over the computation time. Saving the grid values at intermediate time steps significantly reduces the GPUs throughput. As such, it was limited to every 20 time steps (i.e. the computed grid values were saved to the host's disk every 20 iterations). In the case of the FPGA, the grid values could be saved as often as every Ts time steps (i.e. up to the value of fused time steps Ts, which is 5 for this comparison). A $1024 \times 1024 \times 1024$ grid was used with one CUDA kernel launched/dispatched per line along the z-direction (1024 points) for a total of 1,048,576 (1024×1024) kernels. This

resulted in almost 100% utilization of the GPU CUDA cores. The average throughput (from 10 runs) was measured to be ~ 140 GB/s and the average power consumption was 261 W. This translates to ~ 1.86 W/GB/s power efficiency compared to the FPGA implementation with ~ 1.32 W/GB/s power efficiency (From Table 2, FPGA throughput was 35.71 GB/s). This is despite the fact that the GPU is fabricated in a technology node that is two generation ahead of the FPGA's (typically, power efficiency improves by $\sim 30\%$ with each generation [19, 20]). In fact, the GPU power consumption is equivalent to more than 5 FPGAs. If 5 FPGAs were to be used (total power = $48.7 \times 5 \sim 244$ W), the aggregated throughput of the 5 FPGAs would be ~ 179 GB/s or $\sim 27.5\%$ more than that of the GPU.

4.2 Comparison with Published Results

As was mentioned before, the majority of published works focused on 1-D, and 2D stencils or low-order 3D stencils for moderate size grids. Since the main objective of this work is to accelerate high-order 3D stencils on large grids, performance was only compared to published works with similar stencils.

4.2.1 Performance Comparison with Multi-Core CPU Implementations

A multi-core CPU implementation of the same exact 3D stencil used in this work is presented in [3] and [9] using $512 \times 512 \times 512$ and $1024 \times 1024 \times 512$ grid sizes, respectively. Performance was reported in terms of Giga Lattice-Points Updates Per Seconds (GLUPs). [3] used two-sockets Intel Haswell Xeon E5-2698v3 processors (22 nm technology) with 16 cores each running at 2.3 GHz while [9] used a 2.45 GHz 64-core AMD Milan-x CPU (a much more advanced 7 nm technology) with 256 MB L3 Cache. Both did not report power consumptions though. Figure 10 shows the throughput of the three implementations. It shows that the FPGA implementation based on the proposed methodology and using a $512 \times 512 \times 512$ grid size achieved a throughput of 8.93 GLUPs. This throughput is equivalent to ~ 43 Haswell cores or ~ 13 Milan-x cores.

4.2.2 Comparison with Other FPGA Stencil Implementations

In [11] a multi-FPGA framework was proposed to accelerate stencil computations. The authors benchmarked the framework using a 20 nm Xilinx XCKU060 FPGA against earlier FPGA implementations for various 2&3D stencils and reported superior performance. The largest implemented 3D stencil however, was a 6-point only stencil (~ 13 floating point operation per point). The reported throughput and

power efficiency using a single FPGA were 10 GFLOP/s and 8 GFLOP/s/W, respectively. Throughput actually went down for the 2-FPGA implementation. The stencil used in our work is a 25-point stencil requiring about 52 floating point operations per point. Hence a throughput of 8.93 GLUPs is equivalent to ~ 464 GFLOPS/s. With a total power of 48.72W, the power efficiency of the implementation based on our proposed methodology was 9.52 GFLOP/s/W. It should be noted that part of the performance and power improvement is due to the more advance fabrication technology of our FPGA (14 nm technology).

5 Multi-FPGA Methodology Evaluation

As mentioned before, the developed methodology was extended to support two FPGA boards to be attached to two different CPU sockets within the same host. Attaching more than one FPGA per socket won't improve performance as these FPGAs would share the socket's DMA bandwidth for sending and receiving. The 3D grid is divided into two sub-grids, one per socket, and the boundary points are transferred between the two sockets every 5 iterations (as was illustrated in Fig. 8). As was explained before, the number of boundary points that needed to be exchanged between the two FPGAs depended on the stencil size d , the Grid size G , and the number of fused time steps T_s . Table 10 sows the amount of exchanged points for the stencil used throughout this work ($d = 4$) as a function of G and T_s . This table shows that although the relative amount of exchanged boundary points decreases with the grid size (i.e. the exchange time becomes irrelevant as the grid size increases), it is proportional with the T_s . This means as T_s is increased to improve the on-FPGA throughput, the amount of exchanged points also increases, thus diminishing some of the throughput gains.

The overall performance was evaluated using an input grid size of $1024 \times 1024 \times 1024$, a UF of 4 for various tile sizes and values of T_s . The tile's slice size and T_s were varied keeping the 3rd dimension constant at 1,024 points. The FPGAs circuits' performance was simulated using constant latencies (from Table 9) due to the difficulty of attaching two FPGA boards to the available commodity server. Table 11 shows a comparison of the total latencies of computing the whole 3D grid using a single FPGA device and the two FPGA devices. Adding a second FPGA decreases the latency by $\sim 27\%$ (and not 50%) compared to a single FPGA. This non-linear scaling of performance is due to several factors. The first factor is the added latency for exchanging boundary points between the two sockets. As Table 10 shows, the amount of exchanged points increases with the number of fused time steps T_s . Also, the exchange is done via the Quick Path interface (QPI) which has a lower bandwidth compared to that of the Sockets' DDR4 RAMs. The second factor is the



Fig. 10 Throughput comparison between implementation based on the proposed methodology and CPU results for the same stencil reported in [3] and [9]

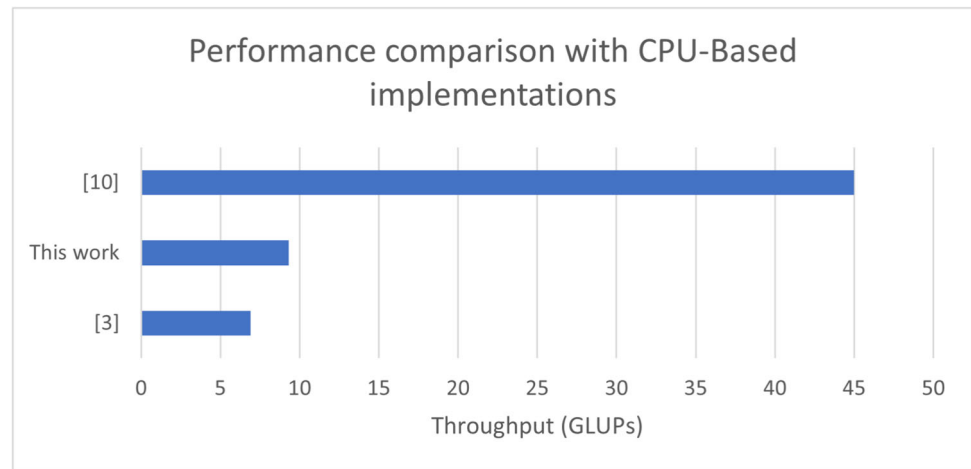


Table 10 Amount of boundary points exchanged between the two FPGAs as a percentage of the total grid size ($G \times G \times G$) as a function of the grid dimension G and T_s ($d = 4$)

G	Ts				
	1 (%)	2 (%)	3 (%)	4 (%)	5 (%)
128	6.26	12.5	18.76	25.0	31.26
256	3.12	6.26	9.38	12.5	15.62
512	1.56	3.12	4.68	6.26	7.82
1,024	0.78	1.56	2.34	3.12	3.90
2,048	0.40	0.78	1.18	1.56	1.96

Table 11 Comparison of total latency (in seconds) of computing the whole 3D grid versus tile slice size and T_s using one and two FPGA devices

Tile size	Ts				
	1	2	3	4	5
Number of devices: 1					
128 × 128	1.04	1.13	1.32	1.44	1.86
256 × 256	1.03	1.04	1.04	1.042	1.06
512 × 512	1.78	1.79	–	–	–
Number of devices: 2					
128 × 128	0.60	0.63	0.82	0.86	1.10
256 × 256	0.71	0.72	0.74	0.76	0.77
512 × 512	1.34	1.35	–	–	–

Table 12 Number of tiles per device versus tile size and T_s for one device and two devices

Tile size	Number of fused iterations (Ts)				
	1	2	3	4	5
Number of devices: 1					
128 × 128	90	100	110	121	156
256 × 256	25	25	25	25	25
512 × 512	9	9	9	9	9
Number of devices: 2					
128 × 128	45	50	60	66	84
256 × 256	15	15	15	15	15
512 × 512	6	6	6	6	6

added synchronization associated with the points exchange. Finally, another major factor for reducing the latency and throughput gains is tiles fragmentation. Since the sub-grids have half the dimension in the Y-direction, more *boundary* tiles arise (tiles that have less points along the Y-direction). These tiles are still sent to the FPGAs even though most of the computed data points are ignored (i.e. not added to the sub-grid). Table 12 shows the total number of tiles per device for 1 and 2 FPGA devices as a function of tile size and Ts. It shows that for the same grid size and Ts, the total number of processed tiles in the case of 2 devices exceeds that of one device by up to 33%.

6 Conclusions

A methodology for accelerating high-order 3D stencils on multiple FPGAs has been developed and implemented as a fully asynchronous SW-HW pipeline. It utilizes spatial and temporal blocking to compute the stencil over any grid size for any number of iterations regardless of the FPGA's off/on-chip memory capacity. It further addresses other issues that were not addressed by previous works like saving of intermediate grid values, and computing the boundary conditions. The methodology enabled the optimization of the performance under the constraints imposed by the utilized FPGA (size of the off/on-chip memory, and bandwidth of the off-chip memory). Experiments showed that throughput scales perfectly with the number of combined iterations, Ts, (up to the FPGA's on-chip memory capacity) and the unrolling factor (UF) up to the off-chip memory bandwidth. Compared to published results for the same stencil on multi-core CPUs, the throughput of implementation of the proposed methodology was equivalent to ~ 43 Haswell cores or ~ 13 Milan-x cores. The same stencil computation was implemented on an NVIDIA A100 GPU with $5 \times$ global RAM size (40 GB) and $37.5 \times$ bandwidth (~ 1.5 TB/s) as the utilized FPGA. The GPU's throughput was ~ $4 \times$ that of the FPGA but its power efficiency (Watts per GB of throughput) was over 40% worse. The methodology was extended to utilize multiple FPGAs (one per host socket). Simulations showed that due to inter-socket communications (resulting from grid division among the sockets), performance does not scale linearly.

Acknowledgements This work has been funded by Saudi Aramco under Research Contract No. 6600011900. The authors also acknowledge the support of King Fahd University of Petroleum & Minerals.

References

1. Opencl overview - the Khronos group inc. URL: <https://www.khronos.org/opencl/>
2. Chi, Y.; Cong, J.; Wei, P.; Zhou, P.: Soda: Stencil with optimized dataflow architecture. In: 2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), pp. 1–8 (2018) <https://doi.org/10.1145/3240765.3240850>
3. Etienne, V.; Tonellot, T.; Malas, T.; Ltaief, H.; Kortas, S.; Thierry, P.; Keyes, D.: High-performance seismic modeling with finite difference using spatial and temporal cache blocking (2017) <https://doi.org/10.3997/2214-4609.201702324>
4. Waidyasooriya, H.M.; Takei, Y.; Hariyama, M.: Opencl-based FPGA-platform for stencil computation and its optimization methodology. IEEE Trans. Parallel Distrib. Syst. (2017). <https://doi.org/10.1109/TPDS.2016.2614981>
5. Zohouri, H.R.; Artur Podobas, S.M.: Combined spatial and temporal blocking for high-performance stencil computation on FPGAs using Opencl. In: International Symposium on Field-Programmable Gate Arrays, pp. 153–162 (2018) <https://doi.org/10.1145/3174243.3174248>
6. Intel. Quartus prime pro design suite. URL: <https://www.intel.com/content/www/us/en/products/details/fpga/development-tools/quartus-prime.html>
7. Sano, K.; Yamamoto, S.: Multi-FPGA accelerator for scalable stencil computation with constant memory bandwidth. IEEE Trans. Parallel Distrib. Syst. (2014). <https://doi.org/10.1109/TPDS.2013.51>
8. Nacci, A.A.; Rana, V.; Bruschi, F.; Sciuto, D.; di Milano, P.; Beretta, I.; Atienza, D.: A high-level synthesis flow for the implementation of iterative stencil loop algorithms on FPGA devices. In: 2013 50th ACM/EDAC/IEEE Design Automation Conference (DAC), pp. 1–6 (2013) <https://doi.org/10.1145/2463209.2488797>
9. Qu, L.; Ltaief, H.; Keyes, D.: Designing tomorrow's algorithms on today's architectures: A case study with stencil computations using AMD 3D v-cachetm technology. (2022) URL: <https://ce.mse.kaust.edu.sa/hicma/news/designing-tomorrows-algorithms-todays-architectures-case-study-stencil-computations-using-amd>
10. Natale, G.; Stramondo, G.; Bressana, P.; Cattaneo, R.; Sciuto, D.; Santambrogio, M.D.: A polyhedral model-based framework for dataflow implementation on FPGA devices of iterative stencil loops. In: 2016 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), pp. 1–8 (2016) <https://doi.org/10.1145/2966986.2966995>
11. Waidyasooriya, H.M.; Hariyama, M.: Multi-FPGA accelerator architecture for stencil computation exploiting spacial and temporal scalability. IEEE Access (2019). <https://doi.org/10.1109/ACCESS.2019.2910824>
12. Wang, S.; Liang, Y.: A comprehensive framework for synthesizing stencil algorithms on FPGAs using Opencl model. In: 2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC), pp. 1–6. (2017). <https://doi.org/10.1145/3061639.3062185>
13. Koraei, M.; Fatemi, O.; Jahre, M.: DCMI: a scalable strategy for accelerating iterative stencil loops on FPGAs. ACM Trans. Archit. Code Optim. **16**(4), 36 (2019). <https://doi.org/10.1145/3352813>
14. Del Sozzo, E.; Conficconi, D.; Sano, K.: Across time and space: Senju's approach for scaling iterative stencil loop accelerators on single and multiple FPGAs. ACM Trans. Reconfig. Technol. Syst. **17**(2), 28 (2024). <https://doi.org/10.1145/3634920>
15. Implementing 3D Finite Difference Codes on the GPU. NVIDIA GPU Technology Conference, 2009, San Jose. https://www.nvidia.com/content/gtc/documents/1006_gtc09.pdf
16. 3D finite differences in CUDA. <https://github.com/rmn64k/cuda3dfd/tree/master>
17. NVIDIA CUDA Toolkit Release Notes. <https://docs.nvidia.com/cuda/cuda-toolkit-release-notes/index.html>
18. GPU & Accelerator process monitoring tool for AMD, Apple, Huawei, Intel, NVIDIA and Qualcomm. <https://github.com/Syllo/nvtop>



19. International Roadmap for Devices and Systems 2017 Edition - More Moore (PDF), ITRS, 2017, [https://www.semiconductors.org/clientuploads/Research_Technology/ITRS/2015/0_2015_ITRS_2.0_20Executive_Report_\(1\).pdf](https://www.semiconductors.org/clientuploads/Research_Technology/ITRS/2015/0_2015_ITRS_2.0_20Executive_Report_(1).pdf)
20. Liu, M.; Philip Wong, H.-S.: How We will Reach a 1 Trillion Transistor GPU. IEEE Spect. Mag. 28 March 2024, <https://spectrum.ieee.org/trillion-transistor-gpu>

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

